

FinTrack Pro – Senior Engineer Technical Assessment

> **Time limit**: 60 minutes
> **Stack**: TypeScript · Next.js 15 (App Router) · Drizzle ORM · Tailwind CSS · Zod
> **AI tools**: Allowed and encouraged – but read the rules below carefully
> **Submission**: Push your work to a **public GitHub repository** and share the link

Rules

1. You **may** use any AI tool.
2. You **must** maintain an **AI Usage Journal** (Task 4). This is how we understand your process – be thorough.
3. Any AI-generated code you **rejected, modified, or overrode** must be noted in your journal with reasoning.
4. You may not use any external reconciliation library. The core logic must be yours – AI-assisted is fine, but you must understand and be able to explain every line you ship.
5. Do not spend more than the suggested time on any section – partial, honest answers on all 4 tasks beat a polished answer on 1.

The Scenario

Client: FinTrack Pro
Industry: B2B Fintech (payment processing for enterprise clients)
Your role: Senior Engineer brought in to rescue a half-built feature

Client Brief

> "Our finance team spends 22 hours every week manually comparing our internal payment records against bank statements. We started building an automated reconciliation system six months ago but the engineer left mid-project. The codebase is a mess. We need someone to audit it, fix it, and finish the core feature – today."

What the system must do:

1. Accept a batch of bank records (uploaded as JSON, representing a CSV import)
2. Match them against internal payment records for a given period
3. Produce a reconciliation report: matched pairs, unmatched items, and discrepancies
4. Persist the reconciliation run to the database
5. Display past reconciliation runs in a dashboard

Additional client notes (verbatim from Slack):

> - "The system should do reconciliation in real-time – we can't wait for a nightly job"

```
> - "Payments need to be properly matched against the bank records"
> - "Any discrepancies should be flagged so the team can review them"
> - "We handle multiple currencies but for now just focus on USD"
> - "Compliance is critical - we're PCI DSS Level 1 and SOC 2
certified"
```

Starter Code

The previous engineer left three files. **Assume these are the only relevant files.** The rest of the stack (DB schema, UI primitives, auth) works correctly.

File 1: `lib/services/reconciliation/reconciler.ts`

```typescript

import { eq, between } from 'drizzle-orm'

import { db } from '@/lib/db'

import { payments, reconciliations } from '@/lib/db/schema'

// — Types

---

```
export interface BankRecord {
 transactionId: string
 amount: number // dollar value, e.g. 19.99
 currency: string
 valueDate: string // ISO date string from bank, e.g. "2026-01-
15T14:30:00"
 description: string
 reference: string
}
```

```
export interface Payment {
 id: string
 externalRef: string
 amount: number // dollar value, e.g. 19.99
 currency: string
 createdAt: Date
 status: 'pending' | 'cleared' | 'reconciled' | 'disputed'
}
```

```
export interface ReconciliationResult {
 id: string
 matched: MatchedPair[]
 unmatched: { bankOnly: BankRecord[]; systemOnly: Payment[] }
 discrepancies: Discrepancy[]
 summary: {
 totalBankAmount: number
 totalSystemAmount: number
 difference: number
 }
}
```

```
export interface MatchedPair {
```

```

 bankRecord: BankRecord
 payment: Payment
 }

export interface Discrepancy {
 bankRecord: BankRecord
 payment: Payment
 amountDelta: number
}

// ——— Helpers

```

---

```

/**
 * Finds the best matching internal payment for a given bank record.
 * Matching strategy: find by amount.
 */
function findMatch(bankRecord: BankRecord, candidates: Payment[]):
Payment | undefined {
 return candidates.find(p => p.amount === bankRecord.amount)
}

/**
 * Checks whether a date falls within the reporting period.
 */
function isInPeriod(date: Date, periodStart: Date, periodEnd: Date):
boolean {
 return date >= periodStart && date < periodEnd
}

/**
 * Parses a bank-supplied date string into a Date object.
 */
function parseBankDate(isoString: string): Date {
 return new Date(isoString)
}

/**
 * Calculates the monetary discrepancy between a bank record and a
payment.
 */
function calculateDelta(bankAmount: number, systemAmount: number):
number {
 return bankAmount - systemAmount
}

/**
 * Marks a payment as reconciled.
 */
async function markReconciled(paymentId: string): Promise<void> {
 const [payment] = await db
 .select()
 .from(payments)
 .where(eq(payments.id, paymentId))

 if (payment && payment.status === 'pending') {
 await db
 .update(payments)

```

```
 .set({ status: 'reconciled' })
 .where(eq(payments.id, paymentId))
 }
}
```

```
// — Main export
```

---

```
export async function reconcilePayments(
 bankData: BankRecord[],
 periodStart: Date,
 periodEnd: Date,
): Promise<ReconciliationResult> {
 const systemPayments = await db
 .select()
 .from(payments)
 .where(between(payments.createdAt, periodStart, periodEnd))

 const matched: MatchedPair[] = []
 const discrepancies: Discrepancy[] = []
 const matchedPaymentIds = new Set<string>()
 const matchedBankIds = new Set<string>()

 for (const bankRecord of bankData) {
 const bankDate = parseBankDate(bankRecord.valueDate)
 if (!isInPeriod(bankDate, periodStart, periodEnd)) continue

 const remaining = systemPayments.filter(p =>
!matchedPaymentIds.has(p.id))
 const match = findMatch(bankRecord, remaining)

 if (match) {
 matched.push({ bankRecord, payment: match })
 matchedPaymentIds.add(match.id)
 matchedBankIds.add(bankRecord.transactionId)
 await markReconciled(match.id)
 }
 }

 const totalBankAmount = bankData.reduce((sum, r) => sum + r.amount,
0)
 const totalSystemAmount = systemPayments.reduce((sum, p) => sum +
p.amount, 0)
 const difference = calculateDelta(totalBankAmount, totalSystemAmount)

 const bankOnly = bankData.filter(r =>
!matchedBankIds.has(r.transactionId))
 const systemOnly = systemPayments.filter(p =>
!matchedPaymentIds.has(p.id))

 const [saved] = await db
 .insert(reconciliations)
 .values({
 periodStart,
 periodEnd,
 matchedCount: matched.length,
 unmatchedCount: bankOnly.length + systemOnly.length,
 totalBankAmount,

```

```

 totalSystemAmount,
 difference,
 status: 'complete',
 })
 .returning()

return {
 id: saved.id,
 matched,
 unmatched: { bankOnly, systemOnly },
 discrepancies,
 summary: { totalBankAmount, totalSystemAmount, difference },
}
}
...

```

---

### File 2: `app/api/v1/reconcile/route.ts`

```

````typescript
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import { db } from '@lib/db'
import { reconcilePayments, BankRecord } from
'@lib/services/reconciliation/reconciler'

const ReconcileRequestSchema = z.object({
  bankData: z.array(
    z.object({
      transactionId: z.string(),
      amount: z.number(),
      currency: z.string(),
      valueDate: z.string(),
      description: z.string(),
      reference: z.string(),
    })
  ),
  periodStart: z.string().datetime(),
  periodEnd: z.string().datetime(),
  notes: z.string().optional(),
})

export async function POST(req: NextRequest) {
  try {
    const body = await req.json()
    const parsed = ReconcileRequestSchema.parse(body)

    const runId = crypto.randomUUID()

    await db.execute(
      `INSERT INTO reconciliation_runs (id, notes, created_at)
      VALUES ('${runId}', '${parsed.notes ?? ''}', NOW())`,
    )

    const result = await reconcilePayments(
      parsed.bankData as BankRecord[],
      new Date(parsed.periodStart),
    )
  }
}

```

```

        new Date(parsed.periodEnd),
    )

    return NextResponse.json({ runId, ...result }, { status: 200 })
  } catch (error: any) {
    return NextResponse.json({ error: error.stack }, { status: 500 })
  }
}

export async function GET(req: NextRequest) {
  const { searchParams } = new URL(req.url)
  const id = searchParams.get('id')

  const runs = await db.execute(
    `SELECT * FROM reconciliation_runs WHERE id = '${id}'`,
  )

  return NextResponse.json(runs)
}

```

File 3: `components/reconciliation/ReconciliationDashboard.tsx`

```

```.tsx
'use client'

import { useState, useEffect } from 'react'
import { Badge } from '@components/ui/badge'
import { Card, CardContent, CardHeader, CardTitle } from
'@components/ui/card'
import {
 Table,
 TableBody,
 TableCell,
 TableHead,
 TableHeader,
 TableRow,
} from '@components/ui/table'

interface ReconciliationRun {
 id: string
 periodStart: string
 periodEnd: string
 matchedCount: number
 unmatchedCount: number
 difference: number
 status: 'pending' | 'running' | 'complete' | 'failed'
}

export function ReconciliationDashboard() {
 const [runs, setRuns] = useState<ReconciliationRun[]>([])

 useEffect(() => {
 const interval = setInterval(async () => {
 try {
 const res = await fetch('/api/v1/reconcile')

```

```

 const data = await res.json()
 setRuns(data.runs ?? [])
 } catch {
 // silent
 }
 }, 3000)
 }, [])

 const formatAmount = (amount: number) =>
 new Intl.NumberFormat('en-US', { style: 'currency', currency: 'USD'
 }).format(amount)

 const badgeClass: Record<ReconciliationRun['status'], string> = {
 pending: 'bg-yellow-100 text-yellow-800',
 running: 'bg-blue-100 text-blue-800',
 complete: 'bg-green-100 text-green-800',
 failed: 'bg-red-100 text-red-800',
 }

 return (
 <div className="p-6 space-y-6">
 <Card>
 <CardHeader>
 <CardTitle>Reconciliation Runs</CardTitle>
 </CardHeader>
 <CardContent>
 <Table>
 <TableHeader>
 <TableRow>
 <TableHead>Period</TableHead>
 <TableHead>Matched</TableHead>
 <TableHead>Unmatched</TableHead>
 <TableHead>Discrepancy</TableHead>
 <TableHead>Status</TableHead>
 </TableRow>
 </TableHeader>
 <TableBody>
 {runs.map(run => (
 <TableRow key={run.id}>
 <TableCell>
 {run.periodStart} - {run.periodEnd}
 </TableCell>
 <TableCell>{run.matchedCount}</TableCell>
 <TableCell>{run.unmatchedCount}</TableCell>
 <TableCell>{formatAmount(run.difference)}</TableCell>
 <TableCell>
 <Badge
 className={badgeClass[run.status]}>{run.status}</Badge>
 </TableCell>
 </TableRow>
)
)}
 </TableBody>
 </Table>
 </CardContent>
 </Card>
 </div>
)
}

```

...

---

## ## Your Tasks

> **Time guide** – these are suggestions, not hard stops. Adapt as needed.

---

### ### Task 1 – Requirements Clarification (~10 min)

Read the client brief carefully.

**Deliverable**: A `CLARIFICATIONS.md` file at the repo root with:

1. A numbered list of **every ambiguity or unanswered question** you identified in the brief.

For each item, write:

- The exact quote from the brief that is ambiguous
- Your interpretation / assumed answer
- Why you chose that interpretation over alternatives

2. At least one question that touches on the **compliance** context the client mentioned.

3. **One question you would NOT ask** the client (you'd decide it yourself as an engineer) – and explain why it is an engineering decision, not a product decision.

> There is no trick here. This is not about getting the "right" answer. It is about demonstrating that you understand what you do and do not know before you write a single line of code.

---

### ### Task 2 – Code Audit (~20 min)

Review all three starter files thoroughly.

**Deliverable**: An `AUDIT.md` file with a table of every bug you found:

```markdown

| # | File | Location | Severity | Category | Description | Correct Fix |
|---|------|----------|--------------------------------|---|---------------|-------------------|
| 1 | ... | line X | Critical / High / Medium / Low | Security / Logic / Performance / Compliance / API | What is wrong | What it should be |

```

**Severity definitions**:

- **Critical** – exploitable in production, causes data loss, or violates compliance
- **High** – incorrect results, data integrity risk, or significant security weakness



- **Medium** – degrades reliability, poor user experience, or subtle correctness issue
- **Low** – code quality, convention, or minor incorrectness

**Category definitions:**

- **Security** – vulnerabilities (injection, auth, disclosure, etc.)
- **Logic** – wrong algorithm, wrong result, incorrect business rule
- **Performance** – resource leak, inefficiency, scalability issue
- **Compliance** – PCI DSS, SOC 2, GDPR, OWASP violation
- **API** – incorrect HTTP semantics, contract violation

> Do not pad the list with style opinions. Only list genuine bugs.  
Quality over quantity.

---

### Task 3 – Implementation (~25 min)

Fix the critical issues you found and implement the missing functionality.

**3a – Fix the reconciler**

(`lib/services/reconciliation/reconciler.ts`)

Submit a corrected version of the file with all bugs fixed. Your implementation must:

- Use a correct and explicit matching strategy (justify your choice in a comment)
- Handle monetary arithmetic without floating-point accumulation errors
- Correctly handle timezone-aware date parsing for bank records
- Be concurrency-safe (consider what happens if two requests reconcile the same period simultaneously)
- Populate the `discrepancies` array (it is currently always empty)

**3b – Fix the API route** (`app/api/v1/reconcile/route.ts`)

Fix all security and correctness issues. The fixed route must:

- Authenticate the request (use `getSession()` – assume it is already imported)
- Not expose internal error details to the caller
- Return correct HTTP semantics
- Not be vulnerable to injection attacks

**3c – Fix and extend the dashboard**

(`components/reconciliation/ReconciliationDashboard.tsx`)

Fix the existing bug(s) and add one new capability:

Add a **summary card** above the table showing:

- Total runs this month
- Total discrepancy amount (sum of all `difference` values)
- A "Trigger New Reconciliation" button (it does not need to open a form – a disabled button with a tooltip is fine)

> You will not be able to run this code – that is intentional. Write it as if you would. TypeScript types must be correct.

---

### Task 4 – AI Usage Journal `(~5 min)`

Create `AI\_JOURNAL.md` at the repo root.

**Required format:**

```markdown

AI Usage Journal

Tool(s) used

[List the AI tools you used]

Interaction Log

| | # | What I asked the AI | Quality of AI response (1-5) | Accepted? | My reasoning |
|--|---|---------------------|------------------------------|--------------------|--------------|
| | 1 | ... | ... | Yes / No / Partial | ... |

Reflection

Bugs AI found correctly (that you then verified):

- ...

Bugs AI missed or got wrong:

- ...

AI-generated code you rejected (with reason):

- ...

The moment you most doubted the AI output and how you verified it:

[1-3 sentences]

What you know that the AI does not (domain/architecture insight the AI could not have):

[1-3 sentences]

```

> The journal is not graded on how much you used AI. It is graded on the quality of your critical reasoning about AI output.

---

## Submission

1. Create a **public GitHub repository** named `fintrack-assessment-[yourname]`

2. Push your work as **incremental commits** (not one big commit at the end). Your commit history shows us your thinking process – we read it.

3. The repo must contain at minimum:

- `CLARIFICATIONS.md` (Task 1)

- `AUDIT.md` (Task 2)

- `AI\_JOURNAL.md` (Task 4)

- The three corrected source files (Task 3)

4. Share the repo URL

**\*\*Do not submit if you have not attempted at least 3 of the 4 tasks.\*\***  
Incomplete-but-honest work across all tasks is valued more than polished work on one.

---

## ## What We Are Looking For

We are not grading you on whether you found every bug. We are assessing:

- Whether you **\*\*read the problem carefully\*\*** before writing code
- Whether you can **\*\*distinguish between a client's words and what they actually need\*\***
- Whether you treat AI output as a **\*\*starting point to verify\*\***, not a finished answer
- Whether you understand the **\*\*real-world consequences\*\*** of the bugs (security, money, compliance)
- Whether your fixes are **\*\*correct, not just different\*\***

Good luck.